

Алина Гроза

Junior Frontend-разработчик

+37377836740 | mgrom250@gmail.com | <https://github.com/molniagrom> |
<https://www.linkedin.com/in/alina-groza-655422354/>



О СЕБЕ

Frontend-разработчик с практическим опытом создания современных веб-приложений на React, TypeScript и Redux Toolkit. Владею интеграцией REST API с использованием RTK Query, разработкой переиспользуемых компонентных архитектур и созданием адаптивных пользовательских интерфейсов. Выпускница программы Frontend Developer IT-Incubator с 6+ готовыми к деплою проектами, развернутыми на Vercel.

ПРОЕКТЫ

Instagram — социальная платформа для обмена фотографиями | Frontend Developer

Стек: Next.js 16 · React 19 · TypeScript · TanStack Query · Radix UI · react-image-crop · Storybook 10 · Feature-Sliced Design (FSD)

- **Дизайн-система и компоненты:** Создала дизайн-токены (палитра цветов, адаптивные брейкпоинты) и систему SVG-спрайтов с компонентами, задокументированными в Storybook
- **Модуль создания постов:** Спроектировала и построила полноценный многошаговый конвейер создания постов (Загрузка → Обрезка → Фильтры → Публикация) с интеграцией S3 presigned URL, применением фильтров через Canvas API и валидацией изображений
- **Мобильная адаптация:** Адаптировала весь процесс создания постов для мобильных экранов с модальными окнами, слайдером предпросмотра, фильтрами для каждого фото
- **Качество кода и рефакторинг:** Декомпозировала монолитные компоненты в переиспользуемые хуки, устранила ошибки строгой типизации TypeScript (strict), обеспечила безопасную работу с FileList в асинхронных сценариях и исправила проблемы сериализации пропсов в Next.js App Router.

My Kinopoisk — каталог фильмов

Стек: React, TypeScript, Vite, Redux Toolkit, RTK Query, TMDB API v4, Zod, Shadcn UI, MUI

- **Разработка SPA-приложения каталога фильмов:** Разработала SPA-приложение каталога фильмов на React + TypeScript + Vite с интеграцией TMDB API v4: реализовала 7 маршрутов, 16+ API-эндпоинтов, OAuth-аутентификацию и систему избранного
- **Архитектура управления серверным состоянием:** Спроектировала слой управления данными на Redux Toolkit Query (RTK Query): настроила кеширование, теговую инвалидацию, оптимистичные обновления UI с откатом изменений при ошибках, Zod-валидацию API-ответов и debounce фильтров
- **Система фильтрации и infinite scroll:** Реализовала сложную систему фильтрации каталога: 8 вариантов сортировки, фильтр рейтинга, жанры и бесконечную прокрутку через IntersectionObserver с предзагрузкой данных. Разработала переиспользуемые кастомные хуки и обработку дедупликации результатов
- **Компонентная архитектура и рефакторинг:** Выполнила декомпозицию компонентов и создание переиспользуемых модулей: разработала общий SliderShell для каруселей, внедрила Shadcn UI + MUI, поддержку dark/light темы и skeleton-состояния для улучшения пользовательского опыта

Rick and Morty Explorer — база данных персонажей мультвселенной

Стек: *Next.js 16 (App Router) · React 19 · TypeScript · Tailwind CSS 4 · SCSS Modules · Axios · React Hook Form · Zod · Storybook 10*

- **Серверная архитектура и API-слой:** Спроектировала серверный прокси через Next.js Route Handlers для 3 типов ресурсов Rick and Morty API (персонажи, эпизоды, локации), реализовала утилиту `requestWithRetry` с экспоненциальной задержкой и обработкой `AbortController` для отмены запросов при размонтировании
- **Кастомный store и управление состоянием:** Реализовала универсальный React Context-store с обобщённым паттерном `ResourceState<T>` (`beginResource` → `resolveResource` → `rejectResource`) для 7 сущностей, обеспечивающий согласованность загрузки, кеширования и обработки ошибок без сторонних библиотек
- **Система избранного и темизация:** Разработала систему избранного для персонажей, эпизодов и локаций с гидрацией из `localStorage` и `debounce`-сохранением, а также переключение `dark/light` темы с 50+ CSS-переменными на каждую тему через `data`-атрибут
- **Маршрутизация и навигация:** Реализовала 8+ маршрутов включая динамические страницы персонажей `[id]`, каскадные фильтры `genetic-data/anomalies/[species]/[gender]/[status]`, пагинацию с кешированием `totalPages` в Map и предзагрузку превью-картинок персонажей для эпизодов
- **Компонентная библиотека и документация:** Разработала 13 переиспользуемых компонентов (`CharacterCard`, `EpisodeCard`, `LocationCard`, `CustomSelect`, `BurgerMenu`, `Skeleton`, `Pagination`, `ProgressBar`) с полной документацией в Storybook 10, включая `a11y-addon` и автоматическое определение `accessibility`-проблем

MusicFun — музыкальная платформа для управления треками и плейлистами

Стек: *React 19 · TypeScript · Vite 8 · TanStack Query 5 · TanStack Router · openapi-fetch · openapi-typescript · React Hook Form · Zod · React Toastify · Feature-Sliced Design (FSD)*

- **Архитектура и типобезопасный API-слой:** Спроектировала сквозную типизацию на базе `openapi-typescript` — автоматическая генерация типов из OpenAPI-спецификации MusicFun API, типизированный клиент `openapi-fetch` с `middleware` для инъекции `Authorization`-заголовка и автоматическим обновлением `access/refresh` токенов при 401-ошибке через единый `refresh`-промис без конкурентных запросов
- **Серверный прокси и безопасность:** Реализовала Vercel Serverless Function как проксирующий слой между клиентом и бэкендом (`musicfun.it-incubator.app`) с инъекцией API-ключа на сервере, фильтрацией 18+ служебных заголовков и 15-секундным `AbortController`-таймаутом — API-ключ никогда не покидает сервер
- **Управление серверным состоянием:** Спроектировала слой кеширования на TanStack Query с паттерном `query-key factories` для 5 сущностей (треки, плейлисты, артисты, теги, авторизация), глобальным `mutation error handler` с парсингом JSON:API-ошибок и отображением `field-level` и `global-level` ошибок через React Hook Form `setError` и `react-toastify`
- **Маршрутизация и FSD-архитектура:** Организовала приложение по Feature-Sliced Design (`app / features / pages / widgets / shared`) с TanStack Router, обеспечивающим автогенерацию дерева маршрутов и `auto code-splitting` на уровне Vite, включая 6 фичевых модулей (`track`, `playlist`, `auth`, `player`, `artist`, `tag`) с разделением на `api/` и `ui/` слои внутри каждой фичи
- **Фича жизненного цикла треков:** Реализовала полный CRUD-цикл треков — загрузка с обложкой, редактирование метаданных, публикация, удаление обложки и самого трека, лайки — каждый `action` как отдельная под-фича с собственным `mutation` и инвалидацией кеша

ToDoList — менеджер задач с проектами, канбан-доской и календарём

Стек: *React 19 · TypeScript · Vite 6 · Redux Toolkit · RTK Query · Material UI 7 · React Router 7 · React Hook Form · Zod · @dnd-kit · Vitest*

- **Архитектура управления серверным состоянием:** Спроектировала централизованный слой RTK Query с базовым API и 4 дополнительными API-слайсами (`todolists`, `tasks`, `kanbanColumns`, `kanbanTasks`), каждый с собственным `middleware` и `tag-based` инвалидацией; реализовала глобальный `app-slice` с автоматическим отслеживанием состояния загрузки и ошибок через `isPending/isFulfilled/isRejected` matchers по всем `endpoint`-ам
- **Обработка ошибок и формы:** Разработала унифицированный `handleError`-utility, перехватывающий ошибки RTK Query на уровне `baseQuery` с маршрутизацией по HTTP-статусам из тела ответа, с диспатчем в `app-slice` и отображением через `ErrorSnackbar`; реализовала формы авторизации с React Hook Form + Zod-валидацией

- **Проекты и канбан-доска:** Реализовала модуль управления проектами с CRUD-операцией для колонок и задач канбан-доски через отдельные RTK Query API (kanbanColumnsApi, kanbanTasksApi) с drag-and-drop сортировкой на базе @dnd-kit/core + @dnd-kit/sortable
- **Компонентная архитектура и темизация:** Организовала 9 фичевых модулей (auth, todolists, projects, calendar, dashboard, profile, captcha, faq, pages) по Feature-Sliced Design с разделением api/lib/ui внутри каждой фичи; внедрила систему dark/light тем через MUI theme factory с переключением из header и хранением предпочтения в Redux

Персональное портфолио — адаптивный сайт-визитка с анимациями и формой обратной связи

Стек: React 18 · TypeScript · Vite · styled-components · Framer Motion · EmailJS · react-scroll · react-alice-carousel

- **Анимации и производительность:** Спроектировала ленивую загрузку 7 секций через React.lazy + Suspense для минимизации начального бандла; реализовала scroll-based и viewport-triggered анимации через Framer Motion и плавную навигацию между секциями через react-scroll
- **Дизайн-система и темизация:** Разработала собственную дизайн-систему на styled-components с 25+ CSS-переменными для цветов, теней и градиентов, организованными в theme-объект с медиа-брекпоинтами и компонентами-обёртками для консистентного адаптивного layout
- **Контактная форма и интеграции:** Интегрировала EmailJS для отправки сообщений без бэкенд-зависимостей с обработкой состояний (отправка / успех / ошибка), валидацией полей через HTML5 constraints и управлением через useRef + useState; подключила react-alice-carousel для отзывов и SVG-спрайт-систему с hover-эффектами для иконок соцсетей
- **Компонентная архитектура:** Организовала приложение по разделам с переиспользуемыми UI-компонентами и типизированным data-слоем для проектов, соцсетей и контактных данных

НАВЫКИ

Языки:	JavaScript (ES6+), TypeScript
Frontend:	React, Next.js, Redux Toolkit, RTK Query, React Router, TanStack Query
Стилизация:	CSS3, SASS/SCSS, Styled Components, Tailwind CSS
Тестирование:	Vitest, Jest, Storybook
Инструменты:	Git, GitHub, Vite, Webpack, ESLint, Prettier, npm, pnpm
DevOps:	Docker (основы контейнеризации), Vercel (деплой и хостинг)
Методологии:	Agile/Scrum, Git Flow, Code Review, Responsive Design

ОБРАЗОВАНИЕ

Программа Frontend Developer, IT-Incubator, 2025

ЯЗЫКИ

Русский — Родной Английский — A2 (улучшаю)